

Generic Programming module-8

Dr. S. Vinila Jinny

Generic Programming

- It is an approach where generic types are used as parameters in algorithms so that they work for a variety of suitable data types and data structures.

Template

- **Templates in c++ is defined as a blueprint or formula for creating a generic class or a function.**
- **We can create a single function or single class to work with different data types using templates.**
- **C++ template is also known as generic functions or classes.**
- **Templates in c++ works in such a way that it gets expanded at compiler time, just like macros and allows a function or class to work on different data types without being rewritten.**

Types of Templates

- **Class Template**
- **Function Template**

Function Template

- **Function template in c++ is a single function that works with multiple data types simultaneously, but a standard function works only with one set of data types.**

Syntax:

template <class T>

returntype functionname(arguments of type T)

{

//...

}

Function Template

```
#include<iostream>
using namespace std;
template<class X>
X func( X a, X b)
{
    return a;
}
int main()
{
    cout<<func(15,8)<<endl;//func(int,int);
    cout<<func('p','q')<<endl;//func(char,char);
    cout<<func(7.5,9.2)<<endl;//func(double,double)
    return 0;
}
```

Function Template

- **Can have any number of argument**
- **Template functions can be overloaded**
- **Can have member function templates**

Function Templates with multiple parameters

Syntax:

```
template<class T1, class T2, class T3,...>  
return type functionname(arguments of types T1, T2,...)  
{  
.....  
}
```

Function Templates with multiple parameters

```
#include<iostream>
#include<string>
using namespace std;
template<class T1, class T2>
void display(T1 x, T2 y)
{
    cout<<x<<" "<<y<<"\n";
}
int main()
{
    display(1999, "EBG");
    display(12.34, 1234);
    return 0;
}
```

Overloading of Template Functions

- **Can be overloaded by template functions or ordinary functions.**
- **Overloading resolution:**
 - Call an ordinary function that has an exact match
 - Call a template function that could be created with an exact match.
 - Try normal overloading resolution to ordinary functions and call the one that matches.
 - An error is generated if no match is found.

Overloading of Template Functions

```
#include<iostream>
using namespace std;
template <class T>
void display(T x)
{
    Cout<<"Template display:"<<x<<"\n";
}
void display()
{
    Cout<<"Explicitly display:"<<x<<"\n";
}
Int main()
{
    display(100);
    display(12.34);
    display('C');
    Display();
    return 0;
}
```

Member Function Templates

Syntax:

Template <class T>

return type classname <T> :: function name(arglist)

{

///....function

}

Member Function Templates

```
template<class T>
Class vector
{
    T *v;
    int size;
    public:
    vector(int n);
    vector(T *a);
    T operator*(vector &y);
};
template<class T>
vector<T> :: vector(int m)
{
    v=new T[size=m];
    for (int i=0; i<size;i++)
        v[i]=0;
}
```

```
template<class T>
vector<T> :: vector(T *a)
{
    for (int i=0; i<size;i++)
        v[i]=a[i];
}
template<class T>
T vector<T> :: operator*(vector &y)
{
    T sum=0;
    for (int i=0; i<size;i++)
        Sum+=this->v[i]*y.v[i];
    return sum;
}
```

Example-sort()

```
#include<iostream>
using namespace std;
template<class T>
void bubblesort(T a[], int n)
{
    for (int i=0;i<n-1;i++)
        for(int j = n-1; i<j;j--;
            if(a[j]<a[j-1];
            {
                swap(a[j],a[j-1])
            }
        }
    template <class X>
    void swap(X &a, X&b)
    {
        X temp=a;
        a=b;
        B=temp;
    }
}
```

```
int main()
{
    int x[5] = {10,50,30,40,20};
    float y[5] = {1.1,5.5,3.2,4.6,7.2};
    bubblesort(x,5);
    bubblesort(y,5);
    cout<<"Sorted x-array:";
    for(int i=0;i<5;i++)
        cout<<x[i]<<" ";
    cout<<endl;
    cout<<"Sorted y-array:";
    for(int j=0;j<5;j++)
        cout<<y[j]<<" ";
    cout<<endl;
    return 0;
}
```

Non type Template arguments

- Syntax:

```
template<class T, int size>
```

```
class array
```

```
{
```

```
    T a[size];
```

```
    ...
```

```
};
```

```
array<int,10> a1;
```

```
array<float,5> a2;
```

```
Array<char,20> a3;
```

Class Template

The class template in c++ is like function templates.

They are known as generic templates.

They define a family of classes in C++.

```
template<class Ttype>
```

```
class class_name
```

```
{
```

```
//class body;
```

```
};
```

Object creation:

```
Classname<type> objectname(arglist);
```

Class Template

```
#include<iostream>
using namespace std;
template <class T>
class XYZ{
private:
    T a,b;

public:
    XYZ(T x,T y)
    {
        a=x;
        b=y;
    }

    void show()
    {
        cout<<"The Addition of";
        cout<<a<<"and"<<b<<"is";
        cout<<add()<<endl;
    }
}
```

```
T add()
{
    T c=a+b;
    return c;
}

};

int main(){
    XYZ addint(8,6);
    XYZ addfloat(3.5,2.6);
    XYZ
    adddouble(2.156,5.234);
    addint.show();
    cout<<endl;
    addfloat.show();
    cout<<endl;
    adddouble.show();
    cout<<endl;
    return 0;
}
```

Class Templates with Multiple Parameters

- **Syntax:**

```
template<class t1, class t2...>  
class classname  
{  
///.....  
};
```

Class Templates with Multiple Parameters

```
#include<iostream>
Using namespace std;
template<class T1, class T2>
class test
{
    T1 a; T2 b;
public:
    test(T1 x, T2 y)
    {
        a=x;
        b=y;
    }
    void show()
    {
        cout<<a<<"and"<<b<<"\n";
    }
};
```

```
int main()
{
    test <float, int> test1(1.23,123);
    test<int,char> test2 (100,'W');
    test1.show();
    test2.show();
    return 0;
}
```

ADVANTAGES

- Templates are type-safe.
- They are generally considered as an improvement over macros for these purposes.
- Templates avoid some common errors found in code that makes heavy use of function-like macros.
- Both templates and macros are expanded at compile time.
- They are a good way of making generalisations for APIs.

DISADVANTAGE

- Disadvantages of Using Templates in C++
- Many compilers do not support nesting of templates.
- When templates are used, all codes exposed.
- Some compilers have poor support of templates.
- Approx all compilers produce unhelpful, confusing error messages when errors are detected in the template code.
- It can make it challenging to develop the template.